CODEASTRA 2.0 | RGIT | CLOUD COMPUTING AND DISTRIBUTED SYSTEMS | PS2

SyncShield

Project explainer and defence guide

Version 2: every feature, including scaling, rate limiting and identity

"Regions can die. Shoppers won't notice. No add-to-cart is lost."

What the project is, what every feature does and how it works, how to show it, what is measured and what is not, and the questions a judge is likely to ask.

Team: Akshit Sharma (A) | Rahil (B) | Miti Shah (C) | Simrit (D) | TE Computer Engineering, SPIT Mumbai

Written from the repository as of 13 October 2026 | main at 596a7f9 (all lanes and the joining steps merged) plus the region-card state la

How to use this guide

This is version 2 of the explainer. Version 1 was written before the scaling, rate-limiting and identity work was joined to the main build, so several things it listed as 'not on main yet' are now built, wired and tested. Every number in this document comes from a run recorded in the repository (README, *docs/tr-log.md*, *docs/scale/*, *docs/failover-latency.md*); where something is unfinished or unmeasured, the text says so.

Section	What you get	Read it when
1 What changed since version 1	The old claim next to the new state, and the list of new features	you have the old guide in your head
2 The project in one page	The pitch, the claims we measure, the demo domain	you have 30 seconds
3 The problem and the seven ideas	What goes wrong in ordinary setups; how the design answers it	you must explain the design
4 What we built	Inventory, tests, by the numbers	you must say what exists
5 Core features	E1 to E15 with proof	you are asked 'does it do X?'
6 The new features in depth	Rate limit, region limit, power, Bengaluru, home regions, burst and scaling tab, sign-in, one-attribute proof, revoked-credential rule, reset	you present the new work
7 Console guide	Every control on the demo page and what it does	you drive the demo
8 Tech stack and architecture	Technologies, why, the picture	you draw it on a whiteboard
9 End-to-end flows	A write, a crash, a partition, a limited region, a sign-in, a revoked event	you narrate the demo
10 Conflict policy	One table and a worked example	conflicts come up
11 Security and trust	What an attacker can and cannot do	security comes up
12 Scalability and evidence	What scales, what was measured, where it stops, all tests	scale or proof comes up
13 Honest limits	What we do not claim	before a judge finds it
14 Questions and answers	51 short answers in eight groups	rehearsal
15 Demo scripts, run guide, glossary	Three-minute core demo, five-minute extension, commands, vocabulary	the day before

A rule we follow
Nothing is called implemented until its test passes, and every number shown must come from a real run on the day. If a result is awkward, we show it and explain it. That habit is part of the pitch: the project is about **honest** consistency.

1 What changed since version 1

The table pairs what version 1 said with the state of the repository now. The right-hand column is what you should say.

Topic	Version 1 said	Now
Per-shopper rate limit	contract frozen, limiter and UI not on main yet	**Built and wired**: token bucket per shopper (429), meters and 'Spam as' in the console, 34 limiter tests, fairness properties run against the real limiter, verified in a real browser.
Region write limit	built, needs a hook line in region/main.py	**Live**: *scale.install(app)* is in region/main.py before CORS. 503 overloaded, browser moves to the next region, errors shown stay 0.
Power dial	built, not live in a default run	**Live**: real group commit. Dial sets status.load.batch to 1 / 8 / 32. Commits fall from 300 to 43 to 21 for 300 writes; throughput barely moves on a laptop, and we say so.
Bengaluru	id and join script present	**Joined live**: starts empty, running regions add it without a restart, same digest in 6.2 s with 555 events; killing it loses nothing. The chaos agent no longer lists it as a dead peer while it is not running.

Topic	Version 1 said	Now
Benchmark	vertical and compare 'unsupported'	**Measured** on the demo machine: every mode has numbers (section 12). The console's Scaling tab reads the same file.
Revoked-credential rule	built, wiring pending	**Wired on /v1/carry**: refused as revoked_credential, quarantined, state hash unchanged. Peer replication deliberately keeps valid history (reason in section 6.9).
Sign-in (TRUE credentials)	optional, in the old dashboard	**In the current console**: identity section, adults-only item (403 claim_required until 'over 18' is proven), person-keyed rate limit, crash without re-sign-in, four-row Tamper.
One-attribute proof	not present	**New**: a purchase can require one attribute (age_over_18 or member); only that attribute is disclosed; the session records which claims were proven and nothing else.
Tests	492 passed, 24 skipped	**558 passed, 0 failed** (the Node-dependent sign-in tests ran too). Console: 118. Partner package: 67. Identity panel: 8. Browser test: 14 of 14 (keys mode), 20 of 20 (sign-in mode).
Fuzz F1	227 runs, 558 revoked events refused	**236 examples, 918** events from a revoked credential refused, no divergence.
Type-check	stopped at vite.config.ts	**Fixed**: @types/node added; npm run build passes the type-check.
Demo console	three buttons for categories, fixed-width sidebar, one 'Degraded' label	**New**: category dropdown, draggable sidebar, cart bar no longer covers products, smaller Add button, **Reset everything**, and region cards that show the state the region reports (Isolated, Constrained, Overloaded).

The new features at a glance

- **Limit each shopper**: a token bucket per shopper; one shopper's spam never uses another's allowance.
- **Limit a region**: past N writes a second a region answers 503 and the browser moves to the next region.
- **Power dial**: group commit, low / medium / high = 1 / 8 / 32 writes saved per transaction.
- **Add Bengaluru**: horizontal scaling, a fourth region joins empty and catches up.
- **Home regions and load bars**: each shopper has a first-choice region; each card shows writes per second.
- **Burst and Spread** with timing, and a **Scaling tab** reading the benchmark file.
- **Sign-in with a partner credential**, a **one-attribute proof**, an **identity panel**, and the **revoked-credential rule**.
- **Reset everything**, a draggable sidebar, and region cards that show the real state (Isolated, Constrained, Overloaded) instead of one 'Degraded' label; the chaos agent can also reset one server.

2 The project in one page

SyncShield is a data layer for applications that run in several regions. Each region (Mumbai, Pune, Delhi, and Bengaluru when it is added) keeps its own full copy of the data and accepts writes at all times. When a whole node dies or the network between nodes breaks, the shopper's browser quietly moves to the next node, no acknowledged write is lost, concurrent edits are settled by written safety rules (or a supervisor), and every screen says honestly how far to trust what it shows.

> **The 30-second version (say this)**
> "Most multi-region systems fail in three quiet ways: they lose the last few seconds of writes when a node dies, they let one of two concurrent edits silently overwrite the other, and their dashboards say healthy when they are not. SyncShield fixes those three things. Every region always accepts writes. Every change is a signed, immutable event, so two regions holding the same events always show the same state. The shopper's browser carries the signed events a dead node had not yet copied, so nothing acknowledged is lost. Risky conflicts go to a supervisor instead of being guessed. And a status chip is never greener than the facts. On top of that we can add regions, limit a flooding shopper or a busy region without showing the user an error, and let a shopper prove one fact, such as being over 18, without revealing anything else."

The claims we measure

Claim	Evidence we show
0 errors shown to shoppers while a node is killed or limited	judge-panel 'errors shown' counter; failover latency test T20; browser test
0 acknowledged add-to-carts lost	'acknowledged = present on survivors' counter; kill -9 test on real processes (T17)
No concurrent edit silently overwritten	conflict receipts, the review queue, tests T4, T6, T7
Status is never greener than the facts	UNKNOWN badge; CONVERGED only with equal digests; tests T1, T2, F1
One shopper cannot starve another	429 for the spammer only; fairness properties against the real limiter; per-shopper meters
A shopper proves one fact and nothing else	'what the shopper holds' next to 'what the shop received'; claims recorded in the session

The demo domain. The original problem statement is about disaster-response data (two control rooms keep taking reports while the link between them is down). We kept the engine domain-neutral and demonstrate it on an **online shop**: shoppers Alice, Bob, Chitra, Dev and Esha add products to carts and check out. 'Disaster' means any anomaly that interrupts the normal flow: a node crash, a partition, a slow link, a traffic spike, tampered data. A shop makes the stakes obvious (money, shipping).

3 The problem and the seven ideas

Problem statement (PS2): geo-distributed data replication with graceful degradation. Several regions hold copies of one service's data. The links are unreliable and the regions can fail. The system must keep working, keep data safe, and tell the truth about its own state while things are broken. Two obvious designs fail: one central server (it stops accepting reports exactly when the network breaks) and copying one database over the other later (it silently destroys what the other side wrote).

Gap in ordinary setups	What happens	What the user feels
The last unreplicated seconds	A node dies holding writes no peer has; failover promotes a replica that never saw them	An acknowledged cart item vanishes
Concurrent edits	Two regions edit while cut off; last-writer-wins keeps one and drops the other	Money or shipping decided by a clock
Honesty of status	Dashboards show healthy from a heartbeat, not from the data	A screen that is wrong exactly when it matters
Visible failover	Clients time out and wait for DNS or a load balancer	Errors and spinners during the incident
Flooding and capacity (new)	One noisy client or a spike takes a region down; adding capacity means downtime	Everyone slows down; or the shop stops

The seven ideas

1 Choose availability: AP with strong eventual consistency. Every region always accepts valid writes and staleness is made visible. The guarantee: two regions holding the same set of events derive identical state.

2 Every change is an immutable, signed event. A write appends an event (CREATE, SET, NOTE, RESOLVE) with an id like mum:17 (origin and gap-free sequence number), a hybrid logical clock stamp, a content hash and the origin's Ed25519 signature. The log is append-only.

3 Causality is written down, not guessed from clocks. A write lists in *supersedes* the field values it had seen. Two writes to one field where neither replaced the other are concurrent: a conflict. Clocks only break ties.

4 State is a pure function of the event set. derive() takes a cart's events and returns state, conflicts and the rule used: no I/O, no randomness, no wall clock in a decision. Order, duplicates and relays cannot change the result.

5 Replication is just copying missing events. Pull by a gap-free per-origin watermark, push as a shortcut, browser carry as a third source, all through one idempotent ingest.

6 The client helps with failover and carries data. The browser SDK probes regions, sends every write with an op_id, keeps an outbox of the signed events it was acknowledged, and on failover posts it to the next region before retrying.

7 Status is computed from facts. Levels, cart badges and the global badge come from heads, digests and reachability. An unreachable region is UNKNOWN (grey), never green.

> **An analogy that works**
> Each branch office keeps its own numbered notebook and only ever adds lines; nothing is erased. Each line says which earlier lines it replaces. To know the current state you read all lines by a fixed rulebook, so two offices that have copied each other's lines read exactly the same state in any order. Each line is signed, so a courier carrying lines between offices cannot change one. The new controls are: a door that limits how many lines one person may add per second, a clerk who can save several lines in one trip to the vault, and a way to open a fourth office that copies the notebooks on day one.

4 What we built

A working distributed system, a test suite that attacks it, and browser consoles. Numbers are counted from the repository.

Part	What it is	Where
Region node	FastAPI service per region: API, write path, storage, signing, roles, status. One process, one SQLite file.	region/api, region/store.py, region/main.py
Merge engine (pure)	canonical JSON and hashing, HLC, derive() with policy v1, priority lanes, digests, data-at-risk, convergence badge, admission rules (incl. revoked credential)	region/engine/
Replication and runtime	pull by watermark, push queue with priority, carry and has endpoints, link switch, levels, group commit, region write limit, load figures, runtime config	region/sync/
Rate limit (new)	token bucket per shopper: 429 with headers, counters in the status block	region/api/ratelimit.py
Trust and identity	role gate; TRUE credential sessions; claims, proven attributes, require_claim, revoked lookup	region/trust/, region/auth.py, collaboration/true/
Browser SDK	failover, op_id retries, outbox and carry, probes, home regions, failback after three good checks	frontend-new/src/sdk
Consoles	**frontend-new** (landing page and live console, the demo), web (earlier console), dashboard (first dashboard, wallet sign-in)	frontend-new/, web/, dashboard/
Identity UI (new)	identity section: claim codes, wallet sign-in, held-versus-received panel, revoke	frontend-new/src/identity/
Chaos and tools	chaos agent (kill, start, reset, tamper), Prove-it, load generator (--actor, --spread), benchmark, key generation, TRUE stack and revoke relay	scripts/
Tests	unit, property, fuzz, real-process integration and failover, sign-in convergence, console unit tests, headless-browser test	tests/, frontend-new/tests
Documents	contract v1.2, conflict policy, consistency model, architecture, failover latency, TRUE write-up, trial-and-reward log, hand-offs	docs/, contracts/

By the numbers

Backend	about 4 400 lines of Python in region/: engine 890, sync 1 400, api 950, trust 600, and the rest in store, config, main, auth and migrations; plus the Node partner code and sidecar
Console	about 6 500 lines of TypeScript and TSX in frontend-new/src
Tests	**558 passed, 0 failed** in the last full run (8 min 21 s; the Node-dependent sign-in tests ran too). Console npm test 118 passed; partner package 67 passed, 1 todo; identity panel 8 passed; npm run build passes.
Browser test	keys mode 14 of 14 checks, sign-in mode 20 of 20, headless Chrome against real regions, 13 October 2026
Fuzz F1	236 examples on three signed nodes with retries, partitions and tampering; 918 events from a revoked credential refused; no divergence
Regions	three by default (Mumbai :8001, Pune :8002, Delhi :8003); Bengaluru :8004 joins on request
Known flake	T21 (degradation ladder) is an intermittent start-up race, written up in fault.md; it does not affect data safety

Team and ownership: **A Akshit**: API, storage, contracts, write path, consoles, the rate limiter. **B Rahil**: merge engine, signing, correctness tests, fuzz, benchmark, the revoked-credential rule. **C Miti**: chaos agent, fixtures, partner sign-in, claims, identity panel, documentation. **D Simrit**: replication, carry, runtime, group commit, region limit, Bengaluru, load tooling.

5 Core features

The essentials from the build plan (E1 to E15). The right-hand column is the proof you can point at.

#	Feature	In plain words	Proof
E1	Independent regions	Each region is its own process and SQLite file; a new region joins empty and catches up	T11; Bengaluru and Delhi join empty and converge
E2	Local-first writes	Saved on the nearest node first; 201 only after commit; a retry with the same op_id is applied once	T3, T16, API tests
E3	Signed events	Ed25519 signature by the origin; checked on ingest and on carry; bad ones are quarantined	T10, T18, F1
E4	Zero-loss failover	The browser retries on the next node and carries the signed events a dead node had not copied	T17 on real processes (kill -9)
E5	Chaos agent	Crash or restart real region processes from the UI (and reset everything); inject a tampered event	verified live
E6	Replication	Pull by gap-free watermark plus push; safe against duplicates, gaps and relays	T1, T8, T15
E7	Partition lab	Link up / slow / down per pair, enforced in both directions, from the UI	T15, split-brain test
E8	Conflict policy v1	Per-field rules (highest, most active, earliest, latest) chosen for safety	one test per rule
E9	Review and resolve	A review queue; a supervisor resolves with a reason; a stale resolve gets 409	T6, T7
E10	CLOSE_VS_NEW_INFO	An order placed while a new item was added elsewhere opens a review; status does not change	T4, engine tests
E11	Honest status	Levels, per-cart badges, risk, queues, digests; CONVERGED / UNKNOWN	T1, T2, API tests
E12	Roles	officer, supervisor, admin from the API key (or the verified session); only a supervisor resolves	role tests, 403 paths
E13	Console	Shop on the left, live backend on the right, crash buttons, inspector, Prove it	browser tests, frontend tests
E14	Recovery	kill -9 safe, restart replays the log, an empty database under a known id is refused	T9, T11
E15	Defended model	README, consistency model, study sheet and this guide	documents

Extras beyond the essentials

- **Prove it**: one button cuts a region off, changes the same cart on both sides, heals, then compares event digests and state hashes on every region. As a command: *python scripts/prove_it.py --chaos 3*.
- **Priority lanes**: payment, order placed, cancel, a speed upgrade, a fulfilment centre and any supervisor decision are P0; cart-activity notes are P2 bulk. The push queue sends the lowest lane first.
- **Data-at-risk meter**: how many of a region's own events no peer has acknowledged yet, and how many of those are critical.
- **Degradation ladder**: a region reports HEALTHY, RECOVERING, CONSTRAINED, ISOLATED or OVERLOADED, each measured by the node itself (SYNCING is the global badge). The console shows each as it is.

6 The new features in depth

Everything in this section is new since version 1 (or newly wired). All of it is **off or neutral by default**: limits at 0, power at medium, sign-in off. The default behaviour is byte-for-byte what it was before, which is why the older tests stayed green. All of it was frozen in advance as **contract v1.2** (section 6.10), so four people could build in parallel and join cleanly.

6.1 Limit each shopper (per-shopper rate limit, HTTP 429)

What the judge sees	A 'requests left' meter under each shopper. Press **Spam as Bob**: Bob's meter empties and he is refused while Alice shops normally. The refusal is a calm notice, not an error, and 'errors shown to shoppers' stays 0. Network tab: Off / 5 / 10 / 20.
How it works	A **token bucket per shopper**: a full bucket holds *burst* tokens and gains *refill* per second. Each write costs one token. It is spent in the write path **right after the op_id replay check** and before anything is stored, so a retry is never refused or counted, and a refused write uses no sequence number and stores nothing. An empty bucket answers **429** {error: rate_limited, scope: actor, retry_after_ms} with Retry-After, X-RateLimit-Limit and X-RateLimit-Remaining; accepted writes carry the same headers, which is how the console draws the meters. Reads, carry and peer replication are never limited. The bucket key is one function: the API-key actor, or, in sign-in mode, the verified person's credential id.
Proof	tests/api/test_ratelimit.py (34 tests); fairness properties with Hypothesis: no actor exceeds burst + refill x time, one actor's spam never reduces another's allowance, a replay costs nothing, now run against the real limiter; checked against three real regions: Bob was refused 21 times while Alice's meter stayed full and the region's own refused_actor counter agreed.
What to say	"One noisy shopper cannot starve the others. The limit is per person, replays are free, and a refusal stores nothing, so it can be retried safely."
Honest limit	Buckets are in memory, per region: a shopper who fails over starts with a full allowance on the next region, so it bounds load per region and is not a global quota. A restart empties the buckets. Meters are exact only when each shopper has their own key (VITE_SHOPPER_KEYS).

6.2 Limit a region (write limit, HTTP 503) and the visible reroute

What the judge sees	Network tab: **Limit this region**. The route line changes to "Serving from Pune, Mumbai is at its write limit". Errors shown to shoppers stays 0. Replication into the limited region keeps working.
How it works	A small ASGI middleware (the *RegionGate*) counts write requests under /v1/reports in a one-second window. Over the limit a write gets **503 {error: overloaded, scope: region}** with Retry-After, before it touches the database. It is installed before the CORS middleware so the 503 carries CORS headers and the browser can read it as 'busy', not 'unreachable'. Never limited: reads, /v1/carry, /v1/events/has, /internal (peer replication) and /admin. The browser sends the write to the next region at once and leaves the busy one alone for 2, then 4, then 5 seconds. A region that refused a write in the last two seconds reports the level OVERLOADED.
Proof	tests/integration/test_scale.py on real processes: over the limit a write is 503 with Retry-After, replication into the limited region still works, limit 0 changes nothing. Measured: limit 40 a second, 300 writes at 24 at a time: 55 accepted and 245 refused (503); afterwards all regions held the same 555 events and the same digest.
What to say	"A busy region sheds load politely and the shopper never sees it. Nothing admitted is lost, and replication is never throttled, so the limited region still receives its peers' events."
Honest limit	The count is a one-second window, not a smoothed bucket, and it counts every admitted write request including ones that later fail validation. Extra concurrency did not make a region faster: one Python process on one SQLite file is the ceiling.

6.3 The power dial (group commit)

What the judge sees	A Low / Medium / High dial on each region card, with '8 per batch' beneath it. A burst at each setting shows the commit count falling.
How it works	Every write normally ends in a COMMIT that waits for the disk (synchronous=FULL). With **group commit**, writes that are already queued share one COMMIT: low = 1, medium = 8, high = 32 writes per transaction. Each write still gets its own SAVEPOINT (a failing write is undone alone), runs under the region lock so sequence numbers stay gap-free, and is **acknowledged only after the COMMIT that contains it**. A lone write commits immediately, so nothing waits for company, and a stalled waiter commits the batch itself after 0.25 s. The dial is changed live through POST /admin/config.
Proof	tests/integration/test_scale_groupcommit.py: the same 300 writes at low, medium and high are all present, gap-free and identical on every region. 24 writers, 300 writes: 300 commits at low, 43 at medium, 21 at high.
What to say	"Batching really cuts commits. On this laptop throughput barely moves, because the SSD makes each commit cheap, and we show what the machine did rather than call it a speed-up."
Honest limit	'Vertical' power is a batch-size setting, not real hardware. Measured: 43.5, 37.5 and 39.9 writes a second at low, medium and high. A read on the same connection can see a batch a few ms before its COMMIT.

6.4 Add Bengaluru (horizontal scaling)

What the judge sees	An 'Add Bengaluru' card shows 'not joined'. Click it: a fourth region card appears, fills up from empty, and the convergence badge returns to CONVERGED.
How it works	Bengaluru (id blr, :8004) is in the port table but not started by default. Starting it through the chaos agent launches it with an **empty database**; the agent then tells the running regions about it (the same call as scripts/join_region.py, POST /admin/config add_peer), so **no region restarts**. Each region re-reads its peer list every sync round, so it starts pulling from and pushing to the new member at once. The agent also stops listing Bengaluru as a peer of regions it starts while Bengaluru is not running, so an absent optional region does not make others look isolated. The browser trusts it only after three good health checks.
Proof	test_bengaluru_joins_a_running_mesh_empty_converges_and_loses_nothing; browser test; measured: **6.2 seconds** from start request to the same digest on all four regions (555 events); killing Bengaluru afterwards loses nothing.
What to say	"Capacity is added without stopping anything: the new region joins empty and copies every origin from its peers until digests match."
Honest limit	In sign-in mode the call that tells running regions about a new peer needs an admin session, so start all four regions together there. All regions still share one laptop.

6.5 Home regions and the writes-per-second bars

What the judge sees	Each shopper has a first-choice region (Alice and Bob: Mumbai; Chitra and Dev: Pune; Esha: Delhi). A 'Home regions' switch turns it off. Every region card has a writes-per-second bar and a queue figure, so a burst visibly splits across regions.
How it works	The SDK sends a shopper's writes to their home region first and to the others in the fixed order Mumbai, Pune, Delhi as the fallback. Each region reports status.load = {writes_per_s over the last 3 seconds, power, batch, queue_depth} and status.rate = {limits and refusal counters}. When a region does not report load, the console measures it from the region's own write counter.
Proof	tests/integration (status carries load and rate with the contract keys, /admin/config round-trips); frontend tests of the SDK order.
What to say	"Shoppers are spread across regions by home, so write capacity grows with regions; the bars show the split."
Honest limit	Home assignment is static in the demo; there is no sharding of data, every region still stores every event.

6.6 Burst, Spread and the Scaling tab

What the judge sees	Network tab: **Burst 300**, **Burst 1000** (all to one region) and **Spread 1000** (divided over every region), each timed until all regions agree. The Scaling tab shows the measured comparison and the time it was measured; 'not measured yet' if the file is missing.
How it works	Burst writes real cart notes as fast as the region takes them and measures how long every region needs to reach the same digest. The Scaling tab reads *public/scale-result.json*, produced by *scripts/bench_scale.py* (HTTP only: --vertical, --horizontal, --compare). The simulator's numbers (?mode=mock) are modelled and labelled as such.
Proof	docs/scale/scale-result.json and docs/tr-log.md (reproduce commands included); table in section 12.
What to say	"Three regions did better than one, by about a third. They share one laptop CPU, so this is not three machines' worth; run-to-run variation is large."
Honest limit	Numbers are from one machine with the load generator on it. If three regions had come out slower than one we would have shown that.

6.7 Sign-in with a partner credential (TRUE), in depth

Sign-in is optional and **off by default** (AUTH_ENABLED=true). It adds a verified identity to every write without putting the issuer on the write path.

Step	What happens	Why it is safe
1 Issue	An admin registers a member; the member claims a credential once with a one-time code. The wallet signs the issuer's nonce with its key (holder proof) and the issuer signs a credential bound to that key.	Codes work once and expire; the holder key never leaves the browser (non-extractable); the role is copied from the issuer's table, never from the caller.
2 Challenge	A region's loopback Node sidecar issues a one-time nonce and audience (120 s). The region, not the browser, chooses them.	A presentation made for another nonce or audience is rejected; one try per challenge.
3 Present	The wallet reveals only the claims the challenge asks for: actor id, member, role, credential id (never name, date of birth, address).	SD-JWT selective disclosure: every claim is a separate salted hash.
4 Verify	The sidecar runs the partner's real verification (issuer signature, disclosures, expiry, holder binding, nonce, audience, age), checks status from the cached issuer-signed list, and signs a **regional session token** (5 minutes).	Trusts exactly one issuer key, pinned from a file; never calls the issuer during a login.
5 Use	Every request carries Authorization: Bearer session. Any region verifies it locally with pinned session keys and its cached status list.	No region needs the issuer or another region: a crash logs nobody out; an issuer outage does not stop valid sessions.
6 Revoke	An admin revokes a member; the issuer publishes a signed status snapshot; each region's sidecar refreshes it (5 s by default) and refuses the member's sessions.	Snapshots are re-verified with the pinned key; a rollback to an older snapshot is refused.

Freshness policy (one table, mirrored in Python and TypeScript and tested against the same cases): status obtained within 60 s is **online**; up to 600 s is **cached** (the limit for logins and privileged actions such as resolve); up to 900 s an ordinary write by a known, previously active credential is still allowed and labelled **stale**; beyond that, or with no status at all, it is refused with 503 and Retry-After. The honest label (online / cached / stale) is stored on every signed event.

What changes for the shop: **officers touch only their own carts** (a supervisor reads everything but does not edit a cart that is not theirs), the actor and the cart owner come from the session and not from the request body, an op_id is bound to the actor who first used it, and the role, auth mode and credential id are part of the signed, hashed event.

6.8 Prove one attribute: the restricted product and the identity panel

What the judge sees	An 18+ product in sign-in mode. Without proof the region answers **403 claim_required** and nothing is stored. The identity panel shows two columns: 'what the shopper holds' (every attribute in the credential) and 'what the shop received' (only the proven claim), the session's time left, and a 'Revoke this shopper' button.
How it works	The credential carries two more signed, separately disclosable yes/no attributes, *age_over_18* and *member*. A challenge may ask for **at most one**. The wallet reveals exactly that attribute; the sidecar accepts the four sign-in claims plus, optionally, exactly the asked attribute (anything else is 401 wrong_claims). The session token records two short lists: what was disclosed and what was proven true, and nothing else from the credential. *require_claim(auth, claim)* raises the contract's 403 body; /v1/auth/me returns claims and disclosed. The claim travels inside the session, so it works at another region after a crash, with the issuer down.
Proof	Sidecar tests with real crypto (8 new); tests/convergence unit and real-process tests: a session with age_over_18 passes, one without gets 403, 'disclosed' never contains an attribute that was not asked for, a flipped value is caught, the same session and claim keep working at Pune with Mumbai, the issuer and Pune's sidecar down.
What to say	"The shop learns the fact it needs, 'over 18', and nothing behind it. Hold what you like; share only what is asked."
Honest limit	One attribute per sign-in: a purchase needing both signs in twice. A proven claim lasts as long as the session (5 minutes). An attribute is a fact the issuer asserted, not a check of the person.

6.9 The revoked-credential rule and Tamper's fourth row

Scenario. Bob's credential is revoked. A region that had not yet heard about it may already have signed a write for Bob. That signed event can later be handed to another region by a browser. **Rule** (engine, region/engine/admit.py): after the signature check, if the event has a credential_id and the injected lookup says it is revoked, reject it as *revoked_credential*. The lookup is a parameter (default: never revoked), so the engine stays pure and every older test is unchanged.

- **Where it runs:** on **/v1/carry only**, the untrusted browser channel. The event is quarantined and the state hash does not move.
- **Why not on replication:** a region that refused an event its peers hold would stall its watermark behind it and never reach their digest. Replication keeps history that was valid when written.
- **The cost, stated openly:** a browser cannot rescue an acknowledged write of a member who was revoked meanwhile if its node died first.
- **Why a carrier cannot strip it:** the credential id is inside the signed, hashed event.
- **The lookup:** region/trust/claims.py is_revoked() reads the status snapshot the region already holds (re-verified on every call). Only an explicit 'revoked' counts; no snapshot or an unknown id is 'not revoked'.
- **In the console:** Tamper gains a fourth row in sign-in mode: a real event, validly signed for a shopper whose credential is then revoked, handed over by a different signed-in person. It must be refused and the state must not move.
- **Evidence:** fuzz scenario 'a revoked actor keeps sending' (918 refused, none stored); tests/convergence/test_join.py; browser test: four rows rejected, quarantine +4, state hash unchanged.

6.10 Contract v1.2 in one page

Item	Detail
Settings (per region, all off by default)	RATE_ACTOR_BURST (0 = off), RATE_ACTOR_REFILL_PER_S (1), REGION_WRITE_LIMIT_PER_S (0 = off), POWER (medium: low 1, medium 8, high 32)
Runtime change	POST /admin/config accepts rate_actor_burst, rate_actor_refill_per_s, region_write_limit_per_s, power and add_peer; unknown or invalid values are 422 and nothing is applied
Fourth region	id blr, Bengaluru, port 8004, data/blr.db; in the port table, not started by default
429	{error: rate_limited, scope: actor, retry_after_ms}; headers Retry-After, X-RateLimit-Limit, X-RateLimit-Remaining
503	{error: overloaded, scope: region}; header Retry-After
403	{error: claim_required, claim: age_over_18}
Never refused	reads; an op_id replay (never limited, never counted); carry; peer replication
Status additions	load {writes_per_s, power, batch, queue_depth}; rate {actor_burst, actor_refill_per_s, region_limit_per_s, refused_actor, refused_region}
Identity functions	subject_key(auth), has_claim(auth, claim), is_revoked(credential_id); event provenance: event.credential_id; carry reason: revoked_credential

6.11 Reset everything, a draggable sidebar and other console upgrades

- **Reset everything** (console header, with a confirmation): the chaos agent stops every region, deletes all their databases **together** (so no region is left holding events another has forgotten), starts them again, and the page clears its counters, outbox and session. Everything is at zero and the first new write is mum:1. Takes about 10 seconds.
- **Why not reset one server's database alone:** a region's next sequence number lives in its own database. Wiping one while its peers still hold its events would reuse sequence numbers, which the system refuses on purpose (the empty-database guard). The agent still has a per-server reset that restarts a region clean and heals its links, keeping its data; it is not on the card.
- **Draggable sidebar:** a handle between the shop and the backend panel resizes the shop column; double-click or Home resets it; the width is remembered.
- **Category dropdown** instead of three buttons; **cart bar** placed under the scrolling area so it never covers a product's text; a smaller **Add** button.
- **Region cards show the real state**: Isolated, Constrained and Overloaded are shown as such. Before this fix they were all drawn as one 'Degraded' label, which hid whether a peer was unreachable, a link was slow or the region was refusing writes.
- **Landing page**: the redundant 'Live demo' link was removed; 'Launch demo' remains.

7 Console guide: every control on the demo page

Where	Control	What it does and means
Top bar	Overview / Inspect / Evidence / How it works	Overview is the live system; Inspect adds region cards and the inspector tabs; Evidence is the Prove-it and counters view; How it works links to the landing walkthrough.
Top bar	Motion On/Off	Turns animation on or off. It changes nothing in the system.
Header	Live backend light, 'N of M regions reachable'	Green when all configured regions answer; amber when some do not; red when none.
Header	Convergence badge	CONVERGED only when every configured region answers with the same event digest, state hash and policy version; SYNCING, UNKNOWN, INCOMPATIBLE, DIVERGED otherwise.
Header	Auto-shoppers	Random adds by random shoppers, to show load without clicking.
Header	Reset everything	Wipes all regions together and starts from scratch (asks first).
Shop	Shopper tabs + meter	Choose who shops. The badge is their item count; the bar under the name is 'requests left' when a limit is on.
Shop	Category dropdown	All essentials, Home, Everyday.
Shop	Add / minus	Every tap is a real signed write. Minus is a new event with quantity -1; nothing is deleted.
Shop	Delivery and order options	Delivery speed, fulfilment centre, Check out, Pay, Cancel; Place order closes the cart. Each is a real write and a field in the conflict table.
Shop	Identity section (sign-in mode)	Paste claim codes, sign in, see held vs received, revoke a shopper.
Region card	Serving requests / Replica	Serving is the region taking the shopper's next write.
Region card	Status light and text	The state the region itself reports: Healthy, Catching up, Constrained, Isolated or Overloaded; or Starting, Crashing, Offline, Unknown while the process is coming up, being killed, not answering, or not seen yet (table below).
Region card	events / carts / at risk	Saved changes; carts they build; this region's changes no peer has confirmed (what a crash could lose).
Region card	Writes per second, Queue, Power	Load figures; the dial changes writes saved per transaction.
Region card	Hold to crash / Restart	Really kills the region process; Restart brings it back (amber while it catches up).
Right	Route line	Which region serves this shopper and why, e.g. 'Serving from Pune, Mumbai is down'.
Right	Evidence tiles and latest action	Acknowledged, recovered, not on survivors; the last write as three steps (saved, signed, peers confirmed).
Inspect tabs	Carts, Network, Review, Outbox, Security, Metrics, Scaling, API, Prove it	Network: link up / slow / cut, pause replication, shopper limit, region limit, Burst, Spread, Spam, Tamper. Review: supervisor resolves a conflict. Outbox: signed events the browser still carries. Security: quarantine and carry counters. Scaling: measured table. Prove it: scripted partition then digest comparison.

What a region card says, and what it means

Card says	The region reports	Meaning
Healthy	HEALTHY	every peer is reachable and its events match
Catching up	RECOVERING	peers still hold events this region lacks: normal after a restart or a heal
Constrained	CONSTRAINED	a link to a peer is set to slow
Isolated	ISOLATED	a peer has failed three times or been silent for five seconds: a cut link or a dead peer
Overloaded	OVERLOADED	it refused a write in the last two seconds (its write limit or a full write queue)
Starting	(process launching)	just started and replaying its log; up to 20 seconds
Crashing	(kill issued)	the kill was sent and the process has not stopped answering yet
Offline	(no answer)	the process is not answering; its data is unknown
Unknown	(never seen)	no answer received yet

If a card says Isolated after you reconnect

A link left **cut** in the Network tab stays cut across restarts (it is stored in the database), so heal it. A region that was restarted can say Isolated for a moment before its first pull (the known T21 start-up race). A peer that is listed but not running also counts: the chaos agent no longer lists Bengaluru as a peer of regions it starts while Bengaluru is not running, so an absent optional region does not make the others look isolated. In every one of these states the region still accepts writes.

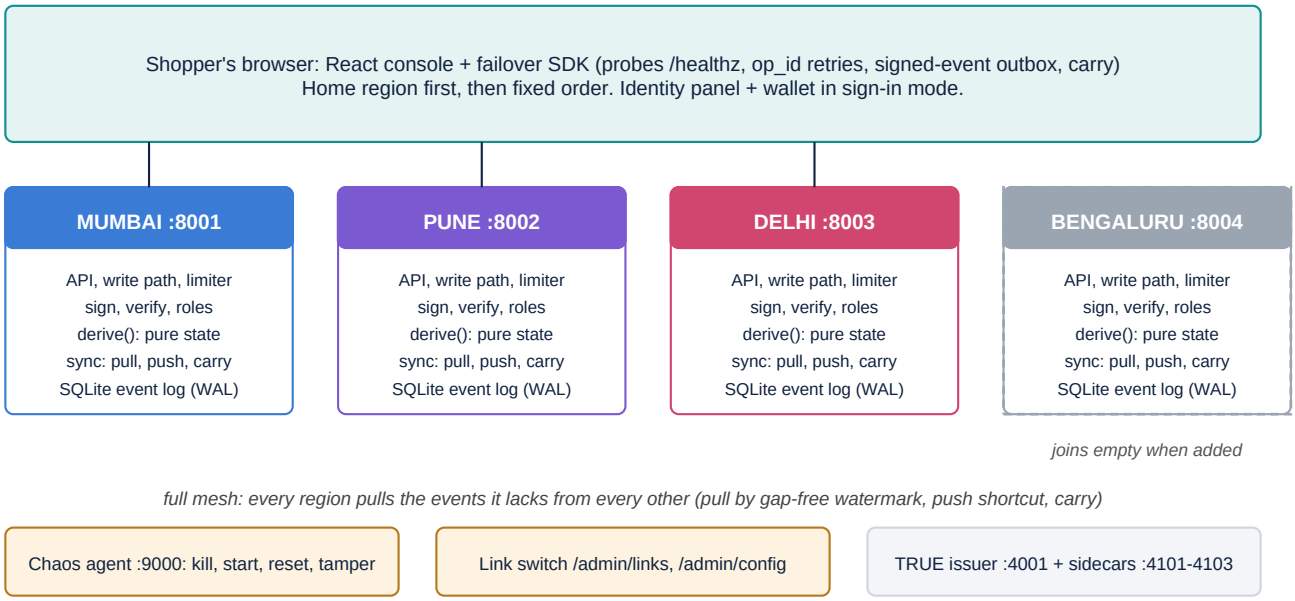
8 Tech stack and architecture

Layer	Technology	Why this one
Region service	Python 3.11+ (tested on 3.14), FastAPI, Uvicorn, Pydantic	Quick to build and test; typed bodies; one worker per region by design (gap-free sequence numbers under an in-process lock)
Storage	SQLite, WAL, synchronous=FULL, one file per region	Real durability (commit means on disk) with no server to run; a region is one file
Write batching (new)	Group commit with per-write SAVEPOINTs	Share one disk wait between queued writes without weakening 'acknowledged after its own commit'
Cryptography	Ed25519 signatures and SHA-256 over canonical JSON	Small, fast, deterministic; canonical JSON gives identical hashes everywhere
HTTP between regions	httpx (async)	Simple timeouts and per-peer control; one client per event loop
Browser	React 18, TypeScript, Vite, react-router	Fast dev loop; typed SDK and contracts; no CDN assets
Identity (optional)	Partner TRUE credentials (SD-JWT VC), Node 24 sidecars, PyJWT sessions	Selective disclosure and revocation without writing any cryptography ourselves
Testing	pytest, Hypothesis (stateful fuzz and fairness properties), vitest, headless-Chrome integration test	Properties and random schedules find bugs examples do not; real-process tests prove kill -9
Tooling	run_regions, chaos_agent, prove_it, loadgen (--actor, --spread), bench_scale, true_stack	One command per scenario, reproducible from the command line

What we deliberately did not use

Not used	Why not
Raft / Paxos / quorum writes	They would make a cut-off region refuse writes; the goal is the opposite (AP). Events merge deterministically, so no agreement on order is needed.
Postgres, Kafka, Kubernetes	More moving parts than the claim needs; SQLite per region already gives durability and a clear failure unit. A scale-out direction, not a feature.
Auto-merging CRDT libraries	They hide the conflict. The problem statement grades a defined conflict policy with a review queue, so the merge logic is ours and visible.
A global rate-limit store	A shared counter would put a coordination point on the write path. Per-region buckets bound the load on each region without one.
Merkle anchoring, log compaction	Out of scope for the build window; listed under limits.

Architecture



Each node is a full stack with its own SQLite file. The browser SDK sends each write to the shopper's home region (or the first region that answers) and fails over to the others. Regions replicate to each other. Bengaluru is a fourth member that joins on request. The chaos agent and the link switch are demo tools, not part of the data path; the TRUE issuer and sidecars are optional and never on the write path.

Component	What it does	The one rule it obeys
Browser SDK	probes /healthz twice a second, retries with the same op_id, keeps the outbox, carries it on failover, fails back after three good probes, leaves a busy region alone 2, 4, 5 s	never shows an error while any node answers
Region API	write path, reads, history, review and resolve, status, runtime config	a write never waits for a peer; 201 only after commit
Rate limiter / region gate	429 per shopper; 503 per region	a refused write stores nothing and uses no sequence number; replays are free
Signing and roles	signs on write; verifies on ingest and carry; checks the role (key or verified session)	a node never signs another node's events
Event log	append-only table keyed by event_id, with an op_id column	never updated, never deleted
derive()	pure function from an event set to state, conflicts and receipts	no I/O, no randomness, no wall clock in a decision
Sync	pull by watermark, push shortcut, carry as a third source; peer list re-read every round	all three use one idempotent ingest
Chaos agent	kill -9, start, reset, tamper; optional regions join without restarting others	admin key only, never on a public deployment

9 End-to-end flows

9.1 The life of one write

Everything in steps 2 to 6 happens inside one SQLite transaction on the serving region:

#	Step	Detail
0	Gates (new)	Region gate: over the region limit is 503. Authenticate (API key or bearer session). op_id replay check: a stored op_id returns the original event with Idempotent-Replay: true and writes nothing. Rate limiter: empty bucket is 429.
1	Check role and op_id	403 before anything is stored if the role may not do this; op_id reuse with a different body is 409.
2	Allocate seq and HLC	Gap-free sequence number and hybrid clock stamp.
3	supersedes and observed	Which field values this write saw and replaces; for status writes, which notes it had seen.
4	Priority lane	P0 / P1 / P2 by the field and value.
5	Hash and sign	SHA-256 over canonical JSON (including role, auth mode, credential id in sign-in mode); Ed25519 signature.
6	Append and derive	Insert the event and refresh the cart view.
7	COMMIT, then 201	With group commit the COMMIT may be shared with other queued writes; the acknowledgement still follows the commit that contains this write. The signed event is in the body and goes to the browser's outbox.

9.2 A node dies: failover and carry

- The browser adds an item (op_id 7f2a); Mumbai answers 201 with signed event mum:17; the outbox holds [mum:17].
- Mumbai is killed (kill -9) before any peer has mum:17. The next add is refused at once (or times out in 600 ms).
- The SDK marks Mumbai down and tries the next region. It **carries the outbox first** (POST /v1/carry: the survivor verifies hash and Mumbai's signature, and, in sign-in mode, that the credential is not revoked), **then retries the add with the same op_id**.
- The shopper sees success; 'acknowledged = present on survivors'. When Mumbai restarts it replays its log, pulls from Pune, and duplicates are ignored by event id: digests equal, CONVERGED.

9.3 A partition, concurrent edits, and the heal

Mumbai and Pune lose their link and both keep writing. Mumbai sets delivery speed 5, centre Bhiwandi and places the order; Pune sets speed 4, centre Chakan and adds a gift card. After the heal both regions show the same answer: speed 5 (highest wins, receipt only); centre Bhiwandi with DOUBLE_DISPATCH (review); status stays 'order placed' with a CLOSE_VS_NEW_INFO review item. A supervisor decides with a reason; that RESOLVE event replicates and the item disappears everywhere.

9.4 A limited region (new)

- Operator switches 'Limit this region' on Mumbai (a low value, so it is easy to exceed) through POST /admin/config.
- Alice's writes (home region Mumbai) exceed it: Mumbai answers 503 overloaded with Retry-After **before touching the database**.
- The SDK treats it as 'busy, not dead': it hands the next region its outbox, retries the write there with the same op_id, and leaves Mumbai alone for 2 seconds, then 4, then 5.
- The route line says why; 'errors shown to shoppers' stays 0; replication into Mumbai continues, so it still receives its peers' events and no digest diverges.

9.5 A sign-in and a restricted purchase (new)

- Esha pastes her one-time claim code; the wallet proves key possession and receives a credential. She signs in: the region's sidecar issues a challenge, the wallet reveals the four sign-in claims (plus 'over 18' when she buys the restricted item), the sidecar verifies and signs a 5-minute session.
- She adds the 18+ item. If she has not proven 'over 18', the region answers **403 claim_required** and stores nothing; the console shows a prompt, not an error. After she proves it, the same add is accepted and the event carries her credential id.
- Mumbai is killed. Her next add succeeds at Pune **with no new sign-in**: Pune verifies the same session locally with a pinned key and its cached status list.

9.6 Reading the status

Signal	Meaning
Region level: HEALTHY, RECOVERING, CONSTRAINED, ISOLATED, OVERLOADED	measured by the node itself: reachability, whether peers' heads differ, a slow link, refused writes. SYNCING is a contract value and the global badge value; a region reports RECOVERING while its peers' heads still differ
Cart badge LOCAL_ONLY / PARTIAL / REPLICATED	how many peers have confirmed every event of this cart
Global CONVERGED	every configured region reachable and identical event digest, state hash and policy version
Global SYNCING	event sets still differ: replication has not finished (normal after a heal)
Global UNKNOWN	a configured region did not answer; never shown as green
Global DIVERGED	same events but different state: derive() is not deterministic. Should never happen; a bug alarm

10 Conflict policy v1 in one table

Applied only when the live values of one field disagree. Writes that saw the old value are updates, never conflicts. The 'why' column is the sentence to say.

Field (shop label)	Rule: value shown until a person decides	Review?	Why it is safe
title, description, category	latest by clock order; the loser stays in history	no	low stakes; nothing is lost
severity (delivery speed)	highest value	receipt only	a lost upgrade is worse than a false one; a later write that saw both can lower it
status (order status)	most active wins: open < assigned < resolved < closed = cancelled; a tie takes the latest	yes	a false 'order placed' can ship an incomplete cart
assigned_team (fulfilment centre)	earliest; flag DOUBLE_DISPATCH when two different centres were named	yes	a stable choice; only a person redirects a shipment
location (delivery pin)	latest; review if candidates are 200 m or more apart	if far	jitter is harmless, a kilometre is not
people_estimate	highest, never summed	yes	20 and 25 for one thing is not 45
notes (cart activity)	union by event id; events with one op_id count once	n/a	adding never conflicts
order placed vs a new item added elsewhere	status stays as derived; a CLOSE_VS_NEW_INFO review item	yes	an addition is never buried under a close
two supervisor decisions that differ	the field's own rule again, flag RESOLVE_VS_RESOLVE	yes	no resolution silently overrides another

Be ready for: 'why does highest win for delivery speed?' Because the error costs are unequal. Showing priority delivery when only same-day was chosen is a small, visible mistake; losing an upgrade the customer paid for is not. The receipt records both values, and a later write that saw both can lower it deliberately.

How the system knows 'the order did not see the item'. Events carry an *observed* list. A status write to closed or cancelled lists every cart note and delivery-speed write its region held; a note or speed write lists the live status

events its region held. If a live close and a note (or a speed of 4 or more) exist where neither lists the other, they were written without seeing each other, and derive() raises the review item. The check uses only the event set, so it is identical on every region.

Admission rules (the other 'rules' in the system)

Rule	Reason code	Applies to
hash does not match the body	bad_hash	ingest and carry
signature does not verify against the origin's key	bad_sig	ingest and carry
origin is not a configured region	unknown_origin	ingest and carry
same event id, different body	id_reused	ingest and carry
validly signed, but the credential has since been revoked (sign-in mode)	revoked_credential	**carry only**

11 Security and trust

The browser is allowed to carry events between regions. That is only safe because of how events are protected.

Attack or accident	What happens	Why it is safe
A browser edits a field of a carried event	hash mismatch, rejected as bad_hash	the hash covers every field except hash and sig
A browser forges an event for another node	bad_sig, rejected	only the origin holds its private key
An event from an unknown region	unknown_origin, rejected	the registry holds only configured regions' keys
The same event carried twice, or also replicated	reported dup; state unchanged	set semantics by event id; derive is idempotent
Same event id, different body	quarantined as id_reused	no silent rewrite of history
A carried event whose predecessors are missing	stored; watermark does not skip the gap; cart hidden as incomplete until the CREATE arrives	the SDK carries the whole unconfirmed outbox in order
A browser withholds events	nothing is lost that a peer already has	it can only fail to help, never harm
A validly signed event from a **revoked** credential	**refused at carry as revoked_credential**, quarantined, state unchanged	the signature binds the credential id, so no carrier can strip it
A forged or replayed sign-in	wrong signature, wrong nonce or a consumed challenge is refused (401 / 409); nothing is issued	partner verification code; one try per challenge
A tampered session token or one signed by an unpinned key	401 (bad signature, wrong issuer, wrong audience, expired)	ES256 only, explicit allowlists, pinned keys
One shopper flooding the system	429 for that shopper only; others unaffected	token bucket per verified person
A spike on one region	503 with Retry-After; browser reroutes	region gate before the database; replication never limited

Roles and sign-in

- **Roles**: officer (create, set, note), supervisor (also resolve), admin (partition lab and runtime settings). A write without the right role is a 403 before anything is stored. In sign-in mode the role comes from the verified session, never from an API key (the legacy key is ignored entirely).
- **Carry has no role check on purpose**: the origin decided authorisation at write time and the signature binds it.
- **Revocation latency**: a revoked member is refused by a region after its next status refresh (5 s default; about 1 s in tests). A region cut off from the issuer keeps its last status for ordinary writes for up to 900 s.

What we do not claim
A region holding a valid signing key can still lie: there is **no Byzantine tolerance**. Localhost traffic is not TLS; a deployed VM needs a TLS reverse proxy. Revocations issued during a partition are unknown to the cut-off region until it reconnects. Peer replication is authenticated by a shared peer token, not by TRUE.

12 Scalability and evidence

12.1 How the design scales

- **Writes scale out.** A write touches only the region it lands on and never waits for a peer, so adding regions adds write capacity as long as shoppers are spread (each has a home region).
- **Reads are local** and cost no coordination.
- **A new region joins empty** and catches up by pulling every origin from any peer, with nothing stopped. Bengaluru: 6.2 s to the same digest with 555 events.
- **Failure units are small**: one region is one process and one file.
- **Overload is shed, not crashed**: the region gate answers 503 early; a shopper's flood is bounded by their bucket.

12.2 What we measured

Latest benchmark (docs/scale/scale-result.json, measured 13 October 2026, 06:11 IST; the file records the machine as 'Windows 11, Intel64 Family 6 Model 186, 12 logical CPUs, Python 3.14.2'; 300 add-to-cart writes, 8 concurrent senders; all regions and the load generator on that one machine):

Setup	Writes per second	p50	p95
One region, low power	43.5	131 ms	148 ms
One region, medium power	37.5	132 ms	179 ms
One region, high power	39.9	121 ms	165 ms
1 region (low power)	39.0	126 ms	158 ms
2 regions (low power)	47.2	88 ms	137 ms
3 regions (low power)	51.5	66 ms	159 ms
One strong region (high) vs three ordinary (low)	38.9 vs 50.6	121 vs 70 ms	146 vs 130 ms

An earlier run by lane B on a different, quieter machine (AMD64 Family 23 Model 24, 4 logical CPUs, 04:22 IST), before the power setting existed, measured 59.1, 84.8 and 92.8 writes a second for 1, 2 and 3 regions (p95 178, 159, 153 ms); a repeat gave 59.0, 77.3 and 97.4. Run-to-run variation is large, so the **shape** (rising, sub-linear) is the result, not the absolute figure.

Write spike and group commit (docs/tr-log.md)

Run	Offered	Accepted	Refused	Per second	p95
One writer at a time (baseline)	200	200	0	73.9	16 ms
24 at once, no limit	300	300	0	63.1	3 703 ms
24 at once, region limit 40 a second	300	55	245 (all 503)	13.4	n/a

- With the limit on, the region's own counter (rate.refused_region) agrees with the 245; all regions then held **555 events** with the same digest and 0 quarantined.
- **More concurrency did not make a region faster** (63 at 24 at once, 74 one at a time): one Python process on one SQLite file is the ceiling here.
- Group commit cuts commits (300, 43, 21) but a laptop SSD makes each commit cheap, so throughput stays about the same.

Failover latency (T20, docs/failover-latency.md, 10 runs each)

Scenario	What happens	p50	p95
Crashed node (kill -9, connection refused)	the SDK is refused at once and retries on the next node	34.9 ms	38.6 ms
Hung node (accepts, never answers)	the SDK waits one request timeout (600 ms) then retries	618.8 ms	631.4 ms

Kill to first success on the survivor: 52 to 61 ms over 4 real-process runs. Plan target: 1.5 s p95.

12.3 Where it stops, and what we would do next

Limit today	Why	What we would do (not built)
Every region stores every event	full replication; no sharding	shard carts by home region; replicate to a few regions
Full mesh: n(n-1)/2 links (3 regions: 3, 4: 6, 10: 45)	every region pulls from every other	gossip or a tree; relay through a hub
One writer per SQLite file; one Uvicorn worker	gap-free sequence numbers under an in-process lock	one writer per shard; a server database per region
Rate-limit buckets are per region, in memory	no coordination on the write path	a shared or gossiped quota if a global limit is needed
The log only grows	append-only by design	snapshots and compaction behind a signed checkpoint
derive() recomputes a cart on each write	pure and simple; cheap for carts of tens of events	incremental views; cache by event-set digest
Pull polls once a second; pages of 200	simple and robust; push covers the common case	long-poll or streaming replication

The one-line answer to 'does it scale?'

"The design has no coordination on the write path, so writes scale with regions, a new region joins empty and catches up, and overload is shed with a 503 instead of failing. On one shared laptop we measured 39, 47 and 52 writes a second for 1, 2 and 3 regions, and 59, 85 and 93 on a quieter run. What does not scale yet is full replication and a full mesh; we know where, and we say it."

12.4 Evidence: tests and measurements

Test	What it proves	Result
T1 idempotent ingest	the same pages twice change nothing (count, digest, state hash)	pass
T2 order independence	all 720 orders of a 6-event fixture give an identical view	pass
T4, T5, T6, T7	split-brain table, disjoint fields, resolve semantics, stale resolve is 409	pass
T8, T15	gap handling; no event crosses a cut link in either direction	pass
T9, T17	kill -9 during writes: every acknowledged id is present; outbox carried; 0 errors	pass, real processes
T16, T18	retry idempotency; forged, edited and unknown-origin events rejected	pass
T20	failover latency, crashed p95 38.6 ms, hung p95 631 ms	measured
F1 fuzz	236 examples, 3 signed nodes, retries, partitions, tampering, a revoked shopper: no divergence; 918 revoked events refused	pass
T21 ladder transitions	slow, cut, heal levels	passed in the last full run; known intermittent (fault.md)
Rate limiter	34 unit tests + fairness properties (no actor exceeds burst + refill x time; spam never hurts another; replay costs nothing) against the real limiter	pass
Scaling on real processes	300 writes at each power: present, gap-free, identical; region limit 503 while replication continues; Bengaluru joins empty and loses nothing when killed	pass
Sign-in convergence	unit tests with throw-away keys + real crypto and real processes: claims, require_claim, crash survival, revocation window, carry refusal, person-keyed limit	pass
Browser test	real console against real regions: keys mode 14 of 14; sign-in mode 20 of 20	pass
Totals	pytest 558 passed, 0 failed; console 118; partner package 67 (+1 todo); identity panel 8; npm run build clean	13 Oct 2026

13 Honest limits

Say these before a judge asks. Stating a limit is part of the pitch.

- **A write is lost only if** its node dies before any peer has it **and** no browser outbox holds it (for example the tab was closed). Pushing critical events first shrinks that window; it cannot remove it.
- **No Byzantine tolerance.** A region holding a valid signing key can still lie.
- **Localhost is not TLS.** A deployed VM needs a TLS reverse proxy.
- **Revocations issued during a partition** are unknown to the cut-off region until it reconnects; a revoked member can be accepted there for a short, stated window (up to 900 s for ordinary writes with a stale snapshot).
- **The revoked-credential rule runs on carry only.** A browser cannot rescue an acknowledged write of a member revoked meanwhile if its node died first; replication keeps such history so digests converge.
- **Power is a batch-size setting on one laptop, not real hardware.** All regions and the load generator share one machine, so 'three regions' is not three machines and numbers vary run to run.
- **Rate-limit counters are in memory, per region**: a limited shopper who fails over starts with a fresh allowance, and a restart empties the buckets. It limits load per region; it is not a global quota.
- **The region limit is a one-second window** and counts every admitted write request.
- **Proof lasts as long as the session** (5 minutes); one attribute per sign-in; an attribute is a fact the issuer asserted.
- **In sign-in mode, start all regions together**: adding a region at runtime needs an admin session.
- **A freshly issued credential** is unknown to a region until its next status refresh (about 5 s).
- **The browser outbox is in memory**: closing the tab clears it. There is no browser offline mode.

- **Nothing is ever deleted from the log**: a removal from the cart is a -1 quantity event.
- **T21** is an intermittent start-up race (written up in fault.md); it does not affect data safety.
- **Not built**: a metrics API (the Metrics tab shows what the status endpoint provides), screenshots of the current UI in the README, link drop-percentage (stored, not applied), WRITE_BATCH_MS and LINK_SLOW_BPS (read, unused).
- **Two machines were used for the benchmarks**: the latest result file records 12 logical CPUs, lane B's earlier run 4, and the trial-and-reward log's header says 4 for all its numbers. Quote a CPU count only together with the run it belongs to.

> **The principle behind all of these**
> Nothing is presented as measured until its test or benchmark has produced it. If three regions had come out slower than one, we would show that and explain why.

14 Questions and answers

Short answers you can say aloud. Each stands alone.

A The problem and the idea

In one sentence, what is SyncShield?

A data layer for apps that run in several regions: every region always accepts writes, nothing acknowledged is lost when a node dies, concurrent edits are settled by written safety rules or a supervisor, and every screen says honestly how far to trust it.

Why a shop, when the problem is about disasters?

'Disaster' means any anomaly: a crash, a partition, a slow link, a spike, tampered data. The engine is domain-neutral; the shop is a skin so the stakes (money, shipping) are obvious.

Isn't this what AWS already does?

Managed services give failover and replication machinery. What happens to the last unreplicated seconds, how concurrent edits are settled and how honestly status is shown are left to the application; defaults typically use asynchronous replication and last-writer-wins. We implement and prove those semantics. (Check provider documentation before quoting specifics.)

Why not one central database with backups?

A central server stops accepting reports exactly when the network breaks, and restoring or copying one database over another silently drops whatever the other side wrote.

What is new here?

Conflicts detected by causality and settled by a written safety policy with a review queue; status never greener than the facts; failover where the browser carries signed events; a defined degradation ladder; and now elastic controls (limit a shopper, limit a region, add a region) and a privacy-preserving identity proof.

B Consistency

Which side of CAP did you choose?

Availability. When the link breaks a region would otherwise have to refuse orders, and refusing is the worse failure. AP with strong eventual consistency.

Why not Raft, Paxos or quorum writes?

They would make a cut-off region refuse writes. We do not need agreement on an order because events merge deterministically: same set in, same state out.

How do you know nodes converge?

derive() is a pure function of the event set. We test every permutation of a fixture (T2), repeated ingest (T1), and a randomized fuzz of three nodes with retries, partitions and tampering (F1), comparing state hashes.

Can a read be stale?

Yes, and the API says so: the cart badge is LOCAL_ONLY, PARTIAL or REPLICATED, and an unreachable region is UNKNOWN.

When does the screen say CONVERGED?

Only when every configured region answers and reports the same event digest, state hash and policy version.

Why do clocks never decide?

Clocks drift. The hybrid logical clock only breaks ties between writes already known to be concurrent; whether a conflict exists is decided by supersedes pointers.

C Conflicts

What counts as a conflict?

Two writes to the same field where neither replaced the other. A write that saw the old value is an update.

Why not last-writer-wins?

It silently drops one concurrent write. We keep every candidate in a receipt.

Can a placed order hide a new item?

No. A new item concurrent with 'order placed' opens a CLOSE_VS_NEW_INFO review; the status stays until a supervisor decides.

What if two supervisors decide differently?

That is a conflict again under the field's own rule, flagged RESOLVE_VS_RESOLVE and reviewed.

What if the supervisor decides on stale information?

A resolve must list the candidates it saw. If the live set changed it gets 409 with the live ids and no event is written.

D Durability, failover and recovery

What does 'acknowledged' mean?

The 201 is returned only after the event is committed on that region's disk and the signed event is in the response. 'Present on survivors' is what the counter checks after a crash.

What exactly can be lost?

An event whose region dies before any peer has it and no browser outbox holds it. We shrink that window and state it.

Could a retried write double an item or a charge?

No. A retry carries the same op_id, so a repeated add is one add. A retried write that was refused with 429 or 503 stored nothing, so retrying is safe.

What happens when the dead node restarts?

It replays its log, reads its next sequence number from storage (never reusing a number), shows RECOVERING, pulls every origin from its peers, and duplicates are ignored. Then digests match.

What if someone wipes one database but keeps the region id?

Forbidden: it would reuse sequence numbers. The run script refuses to start an empty database under an id its peers know. That is why 'Reset everything' wipes all regions together.

How fast is failover?

A crashed node is refused at once, so the retry costs about 35 ms (p95 38.6 ms). A hung node costs one request timeout of 600 ms (p95 631 ms). Plan target: 1.5 s.

E Rate limiting and overload (new)

How do you stop one shopper flooding the system?

A token bucket per shopper: a burst and a steady refill. An empty bucket gets 429 with Retry-After and X-RateLimit headers; nothing is stored and no sequence number is used. Another shopper's allowance is untouched.

Why a token bucket?

It allows a short burst (a real person clicking quickly) and bounds the sustained rate, with a cheap constant-time check and a precise retry_after.

Where in the write path is the limiter, and why there?

After authentication and after the op_id replay check, before the write. So a retry of an already-stored write is never refused or counted, and a refused write leaves no trace.

What is the difference between 429 and 503?

429 means this shopper has used their allowance (scope: actor). 503 means this region is over its write limit (scope: region). The browser treats 429 as a calm notice and 503 as 'move to the next region'.

Is the limit global?

No: buckets are per region, in memory. A shopper who fails over gets a fresh allowance there. It bounds load per region; it is not a quota. A shared store would put coordination on the write path.

Does the limit follow the person or the device?

In sign-in mode it follows the verified person (credential id): two sessions of one person share an allowance. In API-key mode it follows the key.

What does a limited region do to replication?

Nothing: peer replication, reads, carry and admin are never limited, so a limited region still receives its peers' events and digests still converge.

Why does the browser leave a busy region alone for a few seconds?

So a region that is shedding load is not hammered by retries: it is skipped for 2, then 4, then 5 seconds.

F Scalability (new and updated)

How does it scale horizontally?

A write touches only the region it lands on and never waits for a peer, so adding regions adds write capacity if shoppers are spread out (home regions). A new region joins empty and catches up from any peer. Measured on one shared laptop: 39, 47 and 52 writes a second for 1, 2 and 3 regions.

How do you add a region without downtime?

Start it with an empty database. The running regions are told about it through POST /admin/config and re-read their peer list every sync round, so nothing restarts. Bengaluru reached the same digest as the others in 6.2 seconds with 555 events.

And vertically?

A 'power' setting controls how many queued writes share one commit (1, 8 or 32). Commits fell from 300 to 43 to 21, but throughput barely moved on this laptop, so we do not present it as a speed-up. It is a batch-size setting, not real hardware.

Is a write still durable with group commit?

Yes. Each write is acknowledged only after the COMMIT that contains it, each has its own SAVEPOINT, and sequence numbers stay gap-free under the region lock.

What is the bottleneck?

Per region, one writer on one SQLite file and one Uvicorn worker. Across regions, full replication and a full mesh.

Won't a full mesh explode?

Links grow as n(n-1)/2: 3 regions have 3, 10 have 45. Fine for a handful; beyond that we would use gossip or a tree. We say it is a limit.

Is three regions slower than one on your laptop?

Not in our runs (39.0, 47.2, 51.5 and an earlier 59, 85, 93). But all regions and the load generator share the CPUs, and runs vary; we would show it if it came out the other way.

Could it serve a million shoppers?

We do not claim that. We claim a design with no write-path coordination and a small measured run. A real deployment would shard carts by home region with a writer per shard.

G Identity and trust (new and updated)

What does the partner credential add?

A member proves eligibility with a credential; a region issues a short session that every region verifies locally with pinned keys. Roles are enforced on the server and officers touch only their own carts.

What does 'prove over 18' disclose?

Only the yes/no fact 'over 18', plus the minimum sign-in claims (member id, role, credential id). Not a name or a date of birth. The session records which claims were proven and nothing else.

Why does a crash not log anyone out?

The session is verified locally with pinned keys and the cached status list; no region needs the issuer or another region. We tested it with Mumbai, the issuer and Pune's sidecar all down.

How is revocation handled, and how fast?

The issuer publishes a signed status snapshot; each region's sidecar refreshes it (5 s default) and refuses the member's sessions. Measured: about 1 second in tests (0.6 to 1.1 s). A region cut off from the issuer uses its last status within strict limits (600 s for logins and privileged actions, 900 s for ordinary writes).

What does the revoked-credential rule do?

A validly signed event written for a credential that has since been revoked is refused at carry as revoked_credential, quarantined, and the state hash does not move.

Why only on carry, not on replication?

A region that refused an event its peers hold would stall its watermark behind it and never reach their digest. Carry is the untrusted browser channel. The stated cost: a browser cannot rescue an acknowledged write of a member revoked meanwhile.

Can a browser forge or change a carried event?

No. Every event is signed by its origin; the receiver checks the hash and signature, and replays are ignored by event id. A browser can only fail to help; it cannot harm.

What if a region itself is compromised?

We do not claim Byzantine tolerance: a region with a valid key can still lie. That is a stated limit.

H The demo, honesty and process

A region card says Isolated, Constrained or Overloaded. What is the difference?

The card shows the state the region itself reports. Isolated: a peer has failed three times or been silent five seconds (a cut link or a dead peer). Constrained: a link is set to slow. Overloaded: it refused a write in the last two seconds because of its write limit. Catching up: its peers hold events it does not yet, normal after a restart or a heal. In every one of these states the region still accepts writes.

What in the demo is real and what is simulated?

The live console talks to real region processes and the crash button really kills one. A ?mode=mock simulator exists and is labelled as modelled data. Status, priority and conflicts always come from the API.

What did not work or is unfinished?

The T21 start-up race; a metrics API; screenshots in the README; link drop-percentage; the one-attribute proof is one attribute per sign-in. All are listed in this guide.

What does 'Reset everything' do, and why not reset one server?

It stops all regions, deletes all databases together, restarts them, and clears the page. One server cannot be wiped alone because it would reuse sequence numbers, which the system refuses on purpose.

Did AI write this?

AI assistants were used as the event rules allow. Each module's owner explains it line by line, and every pull request was explained back by a teammate. Credits are in the README.

What would you do with another week?

Fix the start-up race, add compaction and shard-by-home-region, make the limiter's quota shared if a global limit is needed, and re-run every benchmark on separate machines.

15 Demo scripts, run guide and glossary

The three-minute core demo

#	You do	They see	You say
1	Add a few items as one shopper	carts fill on all regions; badge LOCAL_ONLY, then PARTIAL, then REPLICATED	"Every add is saved on the nearest node first, then copied. The badge says how far it has spread."
2	Pause replication, add one more, hold Hold to crash on the serving region	the card shakes and goes red; the browser moves to the next region; errors stay 0; the outbox is carried	"Mumbai just died. Shoppers saw nothing. The items it had not copied were carried by their own browsers and verified by signature."
3	Restart the region, resume replication	amber while it catches up, then green; digests equal; no duplicate items	"It replayed its log and pulled what it missed. A retried add counts once."
4	Cut a link, edit one cart on both sides, place the order on one and add an item on the other, heal	conflict chips; side-by-side candidates with the rule in words; CLOSE_VS_NEW_INFO; supervisor resolves; stale resolve shows 409	"We never guess on money or shipping. The safe value is shown, the risky case goes to a supervisor, and the decision replicates."
5	Open Prove it	scripted chaos, then identical digests and state hashes on every region	"Same events in, same state out. Here is the proof, and here is what we do not guarantee."

The five-minute extension: scaling and limits

#	You do	They see	You say
6	Network tab: limit each shopper to 5; press Spam as Bob; then add as Alice	Bob's meter empties and he is refused (calm notice); Alice's meter is full; errors shown stays 0	"Fairness per person: one noisy shopper cannot starve the others."
7	Limit Mumbai, press Spam as Alice (home region Mumbai)	route line: Serving from Pune, Mumbai is at its write limit; errors 0; Mumbai still receives replication	"A busy region sheds load politely; nothing admitted is lost."
8	Click Add Bengaluru	a fourth card joins empty, fills, CONVERGED again	"Capacity added with no downtime: it copies every origin and ends at the same digest."
9	Change the power dial; run Burst 300 at each setting; open the Scaling tab	batch changes 1 / 8 / 32; throughput about the same; measured table	"Batching cuts commits; on a laptop it is not a speed-up, and we say so. Three regions beat one by about a third."
10	(Sign-in mode) paste claim codes, sign in, add the 18+ item, then prove 'over 18'	403 claim_required and nothing stored; after proof the add succeeds; held vs received panel	"The shop learns 'over 18' and nothing behind it."
11	(Sign-in mode) crash the serving region, then Tamper	next add works with no new sign-in; four forged rows rejected, state hash unchanged	"The session is verified locally; a revoked credential's event is refused at carry."

Run guide

What	Command
Install (once)	pip install -r requirements.txt (add -r requirements-auth.txt for sign-in mode) | cd frontend-new; npm install
Crash controller and regions	set REGIONS=mum,pun,del,blr then python scripts/chaos_agent.py and python scripts/run_regions.py mum pun del (chaos agent :9000)
Console	cd frontend-new; npm run dev -> http://localhost:5175/demo (simulator: /demo?mode=mock)
Per-shopper meters	VITE_SHOPPER_KEYS=Alice=dev-alice,Bob=dev-bob,Chitra=dev-chitra,Dev=dev-dev,Esha=dev-esha before npm run dev
Sign-in mode	cd collaboration/true; npm install | python scripts/true_stack.py up --provision | python scripts/true_stack.py relay | set AUTH_ENABLED=true | VITE_AUTH_ENABLED=1 npm run dev
Scripted sign-in demo	python scripts/demo_true.py (starts its own issuer, sidecars and two regions)
Prove it, benchmark, load	python scripts/prove_it.py --chaos 3 | python scripts/bench_scale.py --vertical --horizontal --compare | python scripts/loadgen.py --actor Bob -n 40 / --spread mum,pun,del -n 300
Reset the demo	the console's Reset everything, or stop the regions and delete data/*.db*
Everything tested	pytest | cd frontend-new; npm test; npm run build | cd collaboration/true; npx vitest run

Port	Service
8001 / 8002 / 8003 / 8004	Mumbai / Pune / Delhi / Bengaluru (Bengaluru not started by default)
9000	chaos agent (kill, start, reset, tamper)
5175	frontend-new console (proxies /api/region, /chaos, /issuer, /relay)
4001 / 4101-4103 / 4090	sign-in mode only: issuer / regional sidecars / revoke relay

Glossary

Term	Meaning
event	one immutable change (CREATE, SET, NOTE, RESOLVE) with an id, clock stamp, hash and signature
origin, seq	the region that wrote an event and its gap-free number; the id is origin:seq, e.g. mum:17
op_id	a client-chosen id per operation; the same op_id is applied once, so retries are safe
HLC	hybrid logical clock (milliseconds, counter): used only to break ties and for display
supersedes	the ids of the field values a write had seen and replaces; this carries causality
live value / receipt	a value no write has replaced; the record of a conflict: candidates, the rule, the policy version, whether review is needed
derive()	the pure function from an event set to state, conflicts and receipts
watermark	per origin, the highest seq such that every earlier seq is held; a hole is never skipped
carry, outbox	the browser's list of signed events it was acknowledged, handed to a survivor on failover
quarantine	where an event with a bad hash, bad signature, reused id or revoked credential is kept, never applied
digest	SHA-256 of sorted event ids and hashes (event) or of all derived views (state); equal digests mean equal data
AP, SEC	available and partition-tolerant; strong eventual consistency (same events, same state)
token bucket (new)	a rate limiter that allows a burst and refills at a steady rate; one per shopper here
429 / 503 (new)	this shopper is over their allowance / this region is over its write limit
group commit, power (new)	saving several queued writes in one transaction; each is still acknowledged only after its own commit
home region (new)	a shopper's first-choice region; the fixed order is the fallback
credential, claim (new)	a signed statement the issuer gave a member; a claim is one yes/no or text fact in it
selective disclosure, SD-JWT (new)	revealing only chosen claims of a credential, each separately salted and signed
session (new)	a 5-minute token a region signs after verifying a credential; verified locally by every region
status list / snapshot (new)	the issuer-signed list of credential ids and whether each is active or revoked
sidecar (new)	the loopback Node process beside a region that runs the partner's verification code
require_claim (new)	the check that answers 403 claim_required unless the session proved the needed attribute